Utasítások, változók és típusok, elágazások

Összefoglaló

Utasítások

Egy számítógépes program utasítások sorozatából áll, amelyek lépésről lépésre meghatározzák a program működését. A C# nyelvben az egyszerű utasítások mindig pontosvesszővel végződnek. Az összetett utasítások több egyszerű utasítást vagy bonyolultabb műveletet tartalmaznak, és ezeket kapcsos zárójelek határolják, amelyek a kódblokk kezdetét és végét jelölik ki.

A parancssorra (terminal, console) történő kiírásra használhatjuk a **Console.Write** és **Console.WriteLine** utasításokat. Az utasítás nevét követően zárójelek között kell megadnunk, mit szeretnénk kiírni a képernyőre. Ha például egy rögzített szöveget akarunk kiírni, akkor ezt idézőjelek közé kell tennünk.

```
Console.WriteLine("Hello, World!");
```

Az alapértelmezett bemenetről (ez általában a billentyűzetet jelenti) történő adatbeolvasáshoz használhatjuk a **Console.ReadLine** utasítást.

```
Console.ReadLine();
```

Ha egy utasítás figyelmen kívül hagyását kérjük a végrehajtás során, vagy utasításként nem értelmezhető szöveget szeretnénk a forráskódba írni, úgynevezett megjegyzéseket (kommenteket) használhatunk.

```
// Console.WriteLine("Ez nem lesz kiírva a parancssorban.");
// Ezt sem vesszük figyelembe végrehajtáskor
```

Változók és típusok

Egy programban szinte mindig szükség van adatok tárolására, amit változók segítségével oldhatunk meg. A változók a program futása alatt átmenetileg vagy tartósan tárolják az adatokat. Minden változónak van egy típusa, amely meghatározza, hogy milyen jellegű adatot tárolhatunk benne. A változó használatának első lépése a deklarálás, ahol megadjuk a változó típusát és nevét (azonosítóját). Miután a változót deklaráltuk, a programban ezen a néven hivatkozhatunk rá, és értéket rendelhetünk hozzá, amit értékadásnak nevezünk. Az értékadást az $=$ operátorral végezhetjük el: az operátor bal oldalán álló változó kapja meg az operátor jobb oldalán álló kifejezés értékét. Ezt követően a változó értékét lekérdezhetjük és kiértékelhetjük.

```
int x;     // deklaráció
x = 1;     // értékadás
int y = 2;     // a deklaráció és értékadás összevonható
Console.WriteLine(x);     // kiértékelés
```

A változó típusának megválasztásakor arra kell figyelnünk, hogy az illeszkedjen a tárolandó adathoz. Ez sok esetben egyértelmű, de például egy egész számot eltárolhatunk akár szöveges formában is egy karakterlánc

típusú változóban, miközben valószínűleg nem ez a leghatékonyabb megoldás. A forráskódban megjelenő konkrét értékek, amelyeket közvetlenül a kódba írunk, az úgynevezett literálok. A literálok típusát gyakran az alakjukból lehet felismerni.

```
int egesz = 123;     // 32-bites elöjeles egész
double valos = 1.23456;     // 64-bites lebegöpontos szám
bool logikai = true;     // logikai érték
char karakter = '\n';     // egyetlen karakter (itt épp a sortörés karakter)
string karakterlanc = "Vincent! We happy?";     // szöveg
```

Típuskonverzió, típuskényszerítés

Gyakoran előfordul, hogy adatokat különböző típusok között kell átkonvertálni, hogy a program helyesen működjön. A C# nyelv *erősen típusos*, ami azt jelenti, hogy egy változó értékadásakor csak annak típusával egyező (vagy azzal kompatibilis) értéket rendelhetünk hozzá. A típuskonverzió lehetővé teszi, hogy egy adott típusú adatot valamilyen módon egy másik típusba alakítsunk át (például egy számot szöveggé, vagy fordítva). A típuskonverzió C#-ban két módon történhet.

Implicit típuskonverzióról beszélünk, amikor az átalakítás automatikusan történik, vagyis nem kell külön jeleznünk, hogy típusátalakítást kérünk. Ilyenkor az egyik típus egy másik típusba konvertálható anélkül, hogy az adatvesztés lehetősége fennállna. Erre elgéséges feltétel, ha a konvertálandó típus értéktartománya szűkebb a céltípus értéktartományánál.

```
int egesz = 13;
double valos = egesz;     // a változó automatikusan lebegöpontossá konvertálható
```

Explicit típuskonverzióról beszélünk, amikor a konverzió nem történik meg automatikusan, és a programozónak explicit módon kell jeleznie, hogy átalakítást kér. Ez olyan esetekben szükséges, amikor az átalakítás során fennáll az adatvesztés lehetősége, vagyis a program létrehozója ennek tudatában, a kódja ismeretében kéri az átalakítást. Explicit típuskonverziót, más néven *típuskényszerítést* (typecast) válthatunk ki azzal, hogy a konvertálandó változó vagy érték bal oldalára, kerek zárójelek közé írjuk a céltípus megnevezését.

```
double egyikValos = 13.0;
double masikValos = 13.5;
int egyikEgesz = (int)egyikValos;     // a konvertált érték 13, nincs adatvesztés
int masikEgesz = (int)masikValos;     // a konvertált érték 13, adatvesztés
   történt
```

Szintén gyakori eset, amikor valamilyen adatot szöveges formátumra alakítunk, vagy szöveges adatból szeretnénk kinyerni például numerikus értéket. A **ToString** utasítással bármely értéket szöveges típussá alakíthatunk.

```
double szam = 123.456;
string szoveg = szam.ToString();     // az eredmény "123.456" szöveges formában
char egyBetu = 'A';
string masikSzoveg = egyBetu.ToString();     // az eredmény "A" szövegként
```

Megfelelő formátumú szövegből a benne lévő érték kinyerhető a céltípushoz tartozó **Parse** utasítással. Helytelen formátum esetén (például tizedespont helyett tizedesvesszőt tartalmazó szöveges formátumú valós érték) a program futásidejű hibával áll le.

```
string egeszSzovegkent = "13";
int x = int.Parse(egeszSzovegkent);     // x értéke 13 lesz
string logikaiSzovegkent = "false";
bool y = bool.Parse(logikaiSzovegkent);     // y értéke false lesz
string hibasFormatum = "13 és egy kicsi";
double z = double.Parse(hibasFormatum);     // futásidejü hibát okoz
```

A program vezérlése elágazásokkal

A programkódba írt elágazások használatával döntéseket hozhatunk, és különböző ágakon lévő utasítások végrehajtását indíthatjuk el attól függően, hogy egy adott feltétel igaz vagy hamis. Elágazást az `if` kulcsszóval képezhetünk, amelyet egy zárójelek közé írt logikai kifejezésnek (feltételnek) kell követnie. Amennyiben a feltétel teljesül (igazra értékelődik ki), a feltételt követő utasítás (egyszerű vagy összetett) lefut, ellenkező esetben a végrehajtás átugorja az utasítást. Jó gyakorlat, hogy akkor is összetett utasítást (kódblokkot) adjunk meg, ha az csak egyetlen utasítást tartalmaz, így elkerülhetjük, hogy a szintaktika miatt több utasítás közül csak az első hajtódjon végre, ezzel nem várt működést kiváltva.

Az alábbi kódrészlet például csak abban az esetben írja felül az `ertek` változó tartalmát, ha az nagyobb, mint `maxErtek`.

```
int ertek = 13;
int maxErtek = 100;
if (ertek > maxErtek)
{
    ertek = maxErtek;
}
```

Lehetőségünk van mind a feltétel igaz ágát, mind a hamis ágát kezelni az `else` kulcsszóval. Az alábbi kódrészlet a felhasználó által megadott névtől függően írja ki a a két lehetséges üzenet egyikét.

```
string jatekosNev = Console.ReadLine();
if (jatekosNev == "The Doom Slayer")
{
    Console.WriteLine("Rip and Tear, until it is done.");
}
else
{
    Console.WriteLine("You were nothing but a usurper -- a false idol.");
}
```

Több feltétel ellenőrzésére is lehetőségünk van az `if` és `else` kulcsszavak megfelelő kombinálásával. Ilyen esetben az első igaz feltételt követő utasítás, ennek hiányában a „különben-ág" fut le.

```
int szam = -13;
if (szam < 0)
{
    Console.WriteLine("Negatív érték.");
}
else if (szam == 0)
{
    Console.WriteLine("Nulla.");
}
else
{
    Console.WriteLine("Pozitív érték.");
}
```

Operátorok

Az operátorok olyan speciális szimbólumok vagy kulcsszavak, amelyek egy vagy több operanduson végeznek műveleteket, és egy eredményt adnak vissza. Az operátorok segítségével különböző műveleteket hajthatunk végre, például összeadást, kivonást, logikai műveleteket, értékadást, összehasonlítást. A teljesség igénye nélkül az alábbi operátorokat használjuk leggyakrabban.

Az *aritmetikai operátorokat* matematikai műveletek elvégzésére használjuk. A `+`, `-`, `*`, `/` szimbólumokkal rendre két szám összegét, különbségét, szorzatát és hányadosát képezhetjük. A `%` operátorral maradékképzést (maradékos osztást) valósíthatunk meg. Az operátorok a két operandus típusával egyező eredményt adnak vissza (vagyis például két egész szám hányadosa minden esetben egész számként jelenik meg).

Az *összehasonlító operátorok* két érték összehasonlítására szolgálnak, logikai eredményt adnak vissza. Két érték egyenlőségét a `==` szimbólummal (dupla egyenlőségjellel) vizsgálhatjuk, a nem-egyenlőséget a `!=` szimbólummal. A `<`, `<=`, `>=`, `>` relációs operátorok a megszokott összehasonlításokat valósítják meg.

A *logikai operátorok* két logikai érték kombinálására szolgálnak. A `NEM` logikai operátort a `!` szimbólummal érjük el. A `(!a)` kifejezés értéke pontosan akkor igaz, ha `a` hamis. Az `ÉS` logikai operátort a `&&` szimbólummal érjük el. Az `(a && b)` kifejezés értéke pontosan akkor igaz, ha `a` és `b` egyszerre igaz, minden más esetben a kifejezés értéke hamis. A `VAGY` logikai operátort a `||` szimbólummal érjük el. Az `(a || b)` kifejezés értéke pontosan akkor hamis, ha `a` és `b` egyszerre hamis, minden más esetben a kifejezés értéke igaz.

Változók értékének beállítására használjuk az *értékadó operátort*. A már említett módon értékadást az `a = b;` utasítással valósíthatunk meg, melynek hatására az `a` változó értéke a `b` kifejezés értékét veszi fel.

Ciklusok használata, véletlen számok, matematikai függvények

Összefoglaló

Feltételes ciklusok

A ciklusok lehetővé teszik, hogy egy adott műveletsorozatot többször végrehajtsunk, anélkül, hogy ugyanazokat az utasításokat újra és újra meg kellene írnunk. A ciklus használatakor meg kell adnunk valamilyen feltételt, amelytől függően a ciklushoz rendelt utasítások vagy megismétlődnek, vagy a ciklus véget ér, és a végrehajtás a ciklus után folytatódik.

A *feltételes* ciklus lényege, hogy egy megadott logikai feltétel igaz értéke esetén újra és újra végrehajtjuk a ciklushoz tartozó utasításokat (ezt szokás *ciklusmagnak* is nevezni), amíg a feltétel hamis nem lesz. Ha a feltétel a ciklus elején hamis, akkor a ciklusmag egyszer sem fog lefutni.

Az alábbi kódrészletben nulla kezdőértékkel inicializálunk egy változót, majd azt a ciklusmagban kiírjuk és megnöveljük mindaddig, amíg a változó értéke el nem éri a 10-et.

```
int i = 0;
while(i < 10)
{
    Console.WriteLine(i);
    i = i + 1;
}
```

Végtelen ciklusról beszélünk, ha a ciklusfeltétel soha nem válik hamissá. Ha például egy ciklusfeltétel olyan változótól függ, amelynek értéke a ciklusmag hatására nem változik meg, és a ciklusfeltétel kezdetben igaz, akkor végtelen ciklust kapunk. A végtelen ciklus használata néhány esetben tudatos döntés, például mikrovezérlők[1] esetén általánosan alkalmazott gyakorlat.

A feltétel kiértékelése történhet a ciklusmag előtt (ilyenkor szokás *elöl tesztelő* ciklusról beszélni), illetve a ciklusmagot követően is (ilyenkor *hátul tesztelő* ciklusként szokás rá hivatkozni). Utóbbi esetben a ciklusmag utasításai legalább egyszer végrehajtásra kerülnek, amely bizonyos esetekben hasznos lehet.

Az alábbi példában deklarálunk egy változót, amely azonban nem kap kezdőértéket a ciklus előtt. Mivel `do-while` ciklust használunk, így a ciklusmag legalább egyszer biztosan lefut, ahol viszont a változó értéket kap, tehát a ciklusfeltétel a ciklus végén biztosan kiértékelhető.

```
string input;
do
{
    input = Console.ReadLine();
}
while (input != "stop");
```

[1] Konkrét feladat végrehajtására tervezett, tipikusan apró méretű és egyszerű felépítésű számítógép.

Számláló ciklusok

A *számláló ciklust* olyan esetekben célszerű használni, amikor a ciklusmagot ismert számú alkalommal szeretnénk végrehajtani. Ehhet általában egy számlálót használunk, amelynek értéke minden iteráció után változik, amíg el nem éri a meghatározott határt.

C#-ban a `for` ciklus három tagból áll:

- Inicializátor: ebben deklaráljuk a ciklus változóját, amit kezdőértékkel látunk el. Ez a rész csak egyszer fut le, mielőtt a ciklus elindul.
- Feltétel: ez határozza meg, hogy a ciklusmag hány alkalommal fusson le. Amíg a feltétel igaz, a ciklus folytatódik. A ciklusmag minden lefutása előtt kiértékelődik.
- Iterátor: a ciklusmag minden lefutása után végrehajtódik. Általában a számláló változó növelése vagy csökkentése történik (egyébként tetszőleges műveletet végrehajthatunk).

Az alábbi kódrészlet a `for` ciklus használatára mutat példát, eredménye pontosan megegyezik a fentebb `while` ciklussal már megvalósított nullától tízig számlálással. Az inicializátor részben most is egy változót hozunk létre, ami a nulla kezdőértéket kapja. A feltétel akkor igaz, ha a változó értéke tíznél kisebb. Az iterátor részben a változó értékét növeljük meg eggyel.

```
for (int i = 0; i  < 10; i++)
{
    Console.WriteLine(i);
}
```

Véletlenszám-generálás

Álvéletlen számok alatt olyan, általában számítógéppel előállított számokat értünk, amelyek véletlenszerű mintázatot mutatnak, de valójában egy algoritmus (vagyis egy pontosan ismert, *determinisztikus* szabály) hozza létre őket. Használatuk széleskörűen elterjedt különféle tudományos, mérnöki és gazdasági szimulációs eljárásokban, de ugyanígy kriptográfiai problémákban és videójátékokban is találkozunk velük.

C#-ban véletlen számok generálásához használhatjuk a beépített `Random` típust, amellyel előállíthatunk egész és lebegőpontos számokat is. A véletlen érték előállítása előtt létre kell hoznunk az azokat generáló objektumot az alábbi utasítással.

```
Random generator = new Random();    // a változó neve tetszöleges lehet
```

A véletlen-generátor objektumot ezt követően használhatjuk arra, hogy egyenletes eloszlású[2] véletlen számokat állítsunk elő a segítségével.

Az alábbi példában az első utasítás hatására egy véletlen számot kapunk nulla és a lehetséges legnagyobb 32-bites egész szám között úgy, hogy az előálló érték biztosan kisebb a felső határnál (2^{31}-nél). A második utasításban egy felső határt adhatunk meg, ekkor egy nullánál nem kisebb, de a felső határnál biztosan kisebb értéket kapunk. A harmadik utasítással egy megadott alsó és felső határ közötti véletlen értéket kapunk, ahol az alsó határ előfordulhat a kapott számok között, de a kapott értékek minden esetben kisebbek a felső határnál.

[2] Egy véletlen változót egyenletes eloszlásúnak hívunk, ha bármely lehetséges értéke ugyanakkora eséllyel fordul elő.

```
int x = generator.Next();           // egész szám
int y = generator.Next(13);         // egész szám 0 és 12 között
int z = generator.Next(69, 420);    // egész szám 69 és 419 között
```

Egy 0 és 1 közötti egyenletes eloszlású lebegőpontos értéket kapunk az alábbi utasítással (a kapott értékek most is biztosan kisebbek 1-nél).

```
double randomNumber = generator.NextDouble();
```

Matematikai függvények

C#-ban a matematikai függvényeket a `System.Math` osztályon keresztül érjük el. Az teljesség igénye nélkül megemlítjük a fontosabb aritmetikai műveleteket, mint például a `Math.Abs()` utasítást az abszolútérték kiszámításához, a `Math.Pow()` utasítást a hatványozáshoz, valamint a `Math.Sqrt()` utasítást a négyzetgyök-vonáshoz. A `Math.Log2()`, `Math.Log()` és `Math.Log10()` utasítások rendre a 2-es, természetes és 10-es alapú logaritmusok kiszámítását teszik lehetővé. Az utasítások (függvények) paramétere és az általuk visszaadott érték ugyanolyan típusú, általában egy lebegőpontos érték.

```
double x1 = Math.Abs(-1.23);  // a paraméter és az eredmény típusa lebegöpontos é
    rték
int x2 = Math.Abs(-1);        // a paraméter és az eredmény típusa egész érték
double y1 = Math.Pow(1.2, 3.4);    // kiszámítja 1.2-nek a 3.4-ik hatványát
double y2 = Math.Sqrt(123.4);      // kiszámítja 123.4 négyzetgyökét
double y3 = Math.Log10(42);        // kiszámítja 42 10-es alapú logaritmusát
```

Hasonlóan használhatóak a trigonometriai függvények is. Fontos megemlíteni, hogy ezeknek a paraméterét azonban nem fokban kifejezett szögként, hanem ívmértékként (radiánban kifejezett valós számként) kell megadnunk. A fontosabb matematikai konstansok, mint a π és az e értéke a `Math.PI` és a `Math.E` utasításokkal érhető el. Megjegyezzük, hogy ezek nem függvények, így nem kell zárójeleket kitennünk a nevük után.

```
double z1 = Math.Cos(0);              // kiszámítja 0 rad koszinuszát
double z2 = Math.Sin(Math.PI / 2);    // kiszámítja 90 fok szinuszát
double z3 = Math.Tan(1.23);           // kiszámítja 1.23 rad tangensét
```

Tömbök és listák, bejáró ciklus

Összefoglaló

Tömbök

A tömbök segítségével azonos típusú adatokból tudunk egyszerre többet egyetlen változón keresztül kezelni. A tömb egy olyan gyűjtemény, ahol az egyes elemekre azok sorszámával vagy indexével hivatkozhatunk. Az indexelés a C# programnyelvben – és a programnyelvek többségében – nullától indul.

Hasonlóan a változóknál megismertekhez, első lépésként deklarálnunk kell a tömb elérésére használt változót, és létre kell hoznunk a tömböt, amihez rögzítenünk szükséges a tömb elemeinek típusát, valamint a tömb méretét (a maximálisan elhelyezhető elemek számát). A deklaráció és a tömb létrehozása (inicializáció) elvégezhető egy lépésben. Lehetőségünk van tömb létrehozására az elemeinek felsorolásával is.

```
bool[] myBools;                 // tömb változó deklarációs
myBools = new bool[2];          // tömb létrehozása 2 elemmel
int[] myNums = new int[10];     // a deklaráció és létrehozás összevonható
char[] someChars = { 'a', 'b', 'c' };     // tömb létrehozás az elemek
   felsorolásával
```

Ha nem felsoroljuk a tömb elemeit, hanem egy megadott méretű „üres tömb" létrehozását kérjük, akkor a tömb a típusának megfelelő alapértelmezett értékekkel lesz feltöltve. Ez egész számok esetében 0, lebegőpontos számok esetében 0.0, karakternél '\0' (null karakter), logikai típusnál `false`, referencia típusoknál (pl. string, objektumok) `null` (üres referencia).

Ezt követően értéket adhatunk vagy lekérhetjük egy adott indexen lévő elemét a tömbnek az *indexelő operátor* (szögletes zárójelpár) segítségével.

```
myBools[1] = true;              // tömbelem értékadása
bool someBool = myBools[0];     // tömbelem értékének lekérése
```

A tömb mérete lekérdezhető a `Length` utasítással. A nulla-alapú indexelésből következően a legnagyobb lehetséges index értéke `Length` $- 1$, ezt túllépve indexelési hibával áll le a program futása.

```
int count = myNums.Length;    // count értéke 10 lesz
```

Többdimenziós tömbök

Kétdimenziós tömbök segítségével táblázatos (mátrix-szerű) adatokat tárolhatunk, ahol a tömb egy elemének eléréséhez kettő indexet (nevezhetjük sor- és oszlopindexnek) kell megadnunk. Ezt általánosítva kapjuk a többdimenziós tömböket, ahol egy N-dimenziós tömb egy elemének elérése N darab index segítségével lehetséges.

```
string[,,] mysteriousArray = new string[51, 33, 23];
mysteriousArray[9, 11, 13] = "The Truth is out there, Mulder. But so are lies.";
```

Többdimenziós tömbök esetén a `Length` utasítással a teljes tömb elemszámát kapjuk meg. Ha egy adott dimenziója mentén vett elemszámra van szükségünk, azt a **GetLength** utasítással kaphatjuk meg, amelynek meg kell adnunk, hányadik dimenzióhoz tartozó méretet adja vissza.

Az alábbi kódrészletben először létrehozunk egy üres 11×13-as, egészekből álló kétdimenziós tömböt, ezt követően két változóban eltároljuk a tömb két dimenziójához tartozó elemszámokat (nevezhetjük a sorok és oszlopok számának), majd egymásba ágyazott ciklusokkal előállítjuk az összes lehetséges index-kombinációt, és értéket adunk az elemeknek.

```
int[,] matrix = new int[11, 13];
int numRows = matrix.GetLength(0);    // numRows értéke 11 lesz
int numCols = matrix.GetLength(1);    // numCols értéke 13 lesz

for (int i = 0; i < numRows; i++)
{
    for (int j = 0; j < numCols; j++)
    {
        matrix[i, j] = i + j;
    }
}
```

Listák használata

Szemben a tömbökkel, amelyek méretét (elemszámát) a létrehozáskor meg kell adnunk, és azon később nincs lehetőségünk módosítani, a lista (`List`) egy dinamikus gyűjtemény, amely automatikusan növeli vagy csökkenti a méretét az aktuálisan benne lévő elemek számának megfelelően.

A tömbhöz hasonlóan a lista használatának első lépése is a lista változójának deklarációja, illetve a lista létrhozása (a szintaxis eltér a tömböknél megszokottól). Az alábbi első példában egy egészekből álló üres listát hozunk létre, a második példában három karakterlánccal inicializáljuk a létrehozott karakterláncokból álló listát.

```
List<int> listOfNumbers = new List<int>();
List<string> listOfStrings = new List<string> { "Trust", "No", "One" };
```

A listához további elemeket adhatunk az **Add** utasítással, a **Remove** utasítással pedig törörhetünk belőle egy megadott elemet (annak első előfordulását). Az aktuálisan a listában lévő elemek számát a `Count` utasítással, a teljes lista méretét (kapacitását) pedig a `Capacity` utasítással kérdezhetjük le. A tömbhöz hasonlóan a lista elemei elérhetők indexeken keresztül.

```
List<char> letters = new List<char>();
letters.Add('a');
letters.Add('b');
letters[1] = 'c';
for (int i = 0; i < letters.Count; i++)
{
    Console.Write(letters[i]);
}
```

A `foreach` ciklus

Különféle *gyűjtemények*, például tömbök és listák bejárására használható a `foreach` ciklus. Működésének lényege, hogy automatikusan kezeli a ciklus változójának léptetését, vagyis például egy tömb bejárásánál nem az elemek indexei alapján férünk hozzá azokhoz, hanem a `foreach` ciklus változója közvetlenül a gyűjtemény aktuális elemének értékét veszi fel minden lépésben.

A `foreach` ciklus a gyűjtemény elemeit sorban végigveszi, és minden iteráció során az adott elem értékét hozzárendeli egy ideiglenes változóhoz, amit a ciklusmagon belül használhatunk. Az iteráció addig folytatódik, amíg a gyűjtemény minden elemét be nem jártuk. Fontos megjegyzenünk, hogy a `foreach` ciklus épp ezért nem használható arra, hogy a gyűjtemény elemeit módosítsuk, mert a ciklus változója csak egy másolatot tartalmaz az adott elemről (ezért nem végezhetjük el a ciklusváltozó értékadását).

Az alábbi példában létrehozunk egy tíz elemű tömböt egész számokból, majd a tömböt bejárva számítjuk és kiírjuk az elemeinek négyzetét, amelyet hozzáadunk egy listához is.

```
int[] numbers = new int[] { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
List<int> squares = new List<int>();

foreach (int n in numbers)
{
    int squareNumber = n * n;
    Console.WriteLine($"{n}^2 = {squareNumber}");
    squares.Add(squareNumber);
}
```

Összefoglaló

Karakter típus (`char`) segítségével egyetlen unikód UTF-16 karaktert (betűt, számjegyet vagy speciális jeleket) ábrázolhatunk. Egy karakter típus alapértéke a `'\0'`, érték nélküli `''` (üres) karakter nem értelmezett.

A karakterláncok (`string`) karakterekből (`char`) álló sorozatok, amelyekkel szöveges adatot tárolhatunk. Míg karaktereknél nem lehetséges az üres karakter, addig egy `string` esetén üres karakterláncot is definiálhatunk, ami nem keverendő össze a `null` értékkel. Minden típus karakterlánccá alakítható a **ToString** metódussal, és minden `string` karakterek tömbjévé alakítható a **ToCharArray** metódussal.

```
char aChar = 'a';
char newlineChar = '\n';
string nullString = null;
string emptyString = "";
string question = "Vincent! We happy?";
double pi = 3.14159265359;
string text = pi.ToString();    // "3.14159265359"
```

Műveletek karakterekkel[3]

Az **IsLetter** és **IsDigit** metódusok meghatározzák, hogy a paraméterként megadott karakter betű vagy szám-e. Az **IsLower** és **IsUpper** metódusokkal lekérdezhető, hogy az adott karakter kisbetű, illetve nagybetű-e. A **ToLower** és **ToUpper** metódusok rendre az adott karakter kisbetűs, illetve nagybetűs változatát adják meg.

```
bool isLetter = char.IsLetter('a');    // true
bool isDigit = char.IsDigit('1');      // true
bool isUpper = char.IsUpper('a');      // false
char toLower = char.ToLower('A');      // 'a'
```

Egy `char` típusú érték `int` típusúvá konvertálható, ekkor az adatott karakter *karakterkódját* kapjuk. A karakterkód fordított irányú konverziója egy karaktert eredményez. A karakter mögöttes szám értékéből adódóan egy karaktert lehet *növelni* vagy *csökkenteni* is, vagyis érelmezettek rajtuk a ++ és -- operátorok, ilyenkor valójában a karakterkódjukat módosítjuk.

```
int codeOfA = (int)'A';     // 65
char letterZ = (char)90;    // 'Z'
```

Egy `string` típusú változó egyes karaktereire az indexelő operátorral hivatkozhatunk, hasonlóan a tömböknél megismert módon, ilyenkor a hivatkozott elem egy `char` típusú érték lesz. A karakterlánc hossza a `Length` tulajdonsággal kérdezhető le.

[3] Bővebben a hivatalos dokumentációban: https://learn.microsoft.com/en-us/dotnet/api/system.char?view=net-6.0

```
string quote = "We should have shotguns for this kind of deal.";
char firstChar = quote[0];                  // 'W'
char lastChar = quote[quote.Length-1];      // '.'
```

Műveletek karakterláncokkal[4]

A `string` egy úgynevezett *immutable* (nem módosítható) típus, vagyis tartalmát létrehozás után nem tudjuk elemenként megváltoztatni. Elemenkénti módosítást tudunk elérni, ha a `string`-et előbb karakter tömbbé alakítjuk (**ToCharArray**), a tömb elemein módosítunk, majd visszaalakítjuk a tömböt `string`-gé.

```
string quote = "They speak English in What?";
char[] quoteAsArray = quote.ToCharArray();
quoteAsArray[3] = 'Y';
string modQuote = new string(quoteAsArray);    // "TheY speak English in What?"
```

A **Join** metódussal egy karakterláncot állíthatunk elő valamilyen gyűjtemény (pl. tömb) elemeinek összefűzésével. Az elemek közötti elválasztójel paraméterként adható meg.

```
string[] words = { "Jack", "Rabbit", "Slim's" };
string concatenated = String.Join('-', words);    // "Jack-Rabbit-Slim's"
```

Karakterláncokkal végzett műveletek során hasznosak lehetnek az alábbi metódusok. A metódusok egy része *módosít* a karakterlánc tartalmán, de ilyenkor a módosított változatot visszatérési értékként szolgáltatja, vagyis nem közvetlenül az eredeti karakterláncon végzi el (hiszen az nem módosítható).

Két karakterlánc összehasonlítható *lexikografikusan*, vagyis betűrend szerint a **Compare** függvénnyel: az eredmény 0, ha a két karakterlánc megegyezik, -1 ha az első betűrendben megelőzi a másodikat, és 1 ha az első betűrendben a második után következik.

```
int result = String.Compare("Marsellus", "Jules");    // 1
```

Hasznos lehet egy `string` tartalmának ellenőrzésekor, hogy az üres vagy `null` értékű-e, erre szolgálnak az **IsNullOrEmpty** és **IsNullOrWhiteSpace** metódusok.

```
string nothing = "";
string something = "    ";
bool isEmpty = String.IsNullOrEmpty(nothing);                 // true
bool isEmptyToo = String.IsNullOrWhiteSpace(something);       // true
```

A **Contains** segítségével lekérdezhetjük egy karakter vagy karakterlánc előfordulását (van vagy nincs). Az **IndexOf** és **LastIndexOf** metódusok rendre egy karakter vagy karakterlánc első és utolsó előfordulásának indexét ajdák meg (ha nincs előfordulás, akkor a visszaadott érték -1 lesz).

[4] Bővebben a hivatalos dokumentációban: https://learn.microsoft.com/en-us/dotnet/api/system.string?view=net-6.0

```
string quote = "Oh man, I shot Marvin in the face.";
bool contains = quote.Contains('M');       // true
int firstIdx = quote.IndexOf('I');         // 8
int lastIdx = quote.LastIndexOf("in");     // 22
int noIdx = quote.IndexOf("Pumpkin");      // -1
```

A **ToLower** és **ToUpper** kisbetűssé, illetve nagybetűssé alakítják a szöveget. Az **Insert** és **Remove** metódusokkal index alapján szöveget szúrhatunk a karakterláncba, illetve eltávolíthatjuk egy részét. A **Replace** metódus az első paraméterként megadott karaktereket a második paraméterként megadott karakterekre cseréli. A **Trim** a karakterlánc elején és végén található üres karaktereket (pl. whitespace, tabulátor) távolítja el. A **Substring** az eredeti karakterlánc rész-karakterláncát állítja elő, a megadott kezdőindextől adott számú karakterrel. A **Split** egy tömböt eredményez, amelyet az eredeti karakterlánc feldarabolásával kapunk a megadott a karaktert vagy karakterláncot használva elválasztóként (amely nem jelenik meg az eredményében).

```
string quote = "Big Kahuna Burger";
string lower = quote.ToLower();                    // big kahuna burger
string inserted = quote.Insert(11, "Cheese ");     //Big Kahuna Cheese Burger
string removed = quote.Remove(quote.IndexOf("Kahuna"), 7);     // Big Burger
string replaced = quote.Replace('B', 'K');         // Kig Kahuna Kurger
string substring = quote.Substring(4, 6);          // Kahuna
string[] splitted = quote.Split(' ');              // { "Big", "Kahuna", "Burger"
   }
```

Formázott karakterláncok

Formázott karakterláncokat állíthatunk elő többek között karakterlánc *interpolációval*. Ilyenkor a karakterláncot nyitó idézőjel elé a $ jelet téve, a karakterláncban kapcsos zárójelek közé helyezett változónevek (illetve kifejezések vagy metódushívások) azok értékeivel helyettesülnek. Opcionálisan formázást is beállíthatunk (ezt nem részletezünk, csak egy példát mutatunk rá).

```
double normalTime = 13.0;
int mrWolfeTime = 10;
string expression = $"That's {normalTime:F3} minutes away. I'll be there in
   {mrWolfeTime}."
// "That's 13.000 minutes away. I'll be there in 10."
```

Fájlok és könyvtárak kezelése

Összefoglaló

Fájlok és könyvtárak kezelésére számos eszköz érhető el a `System.IO` névtéren keresztül. A projekt beállításától függően ennek használatba vétele vagy automatikusan megtörténik, vagy manuálisan, a fájl elején meg kell tennünk.

```
using System.IO;
```

Könyvtárak kezelése

A könyvtárak kezeléséhez szükséges műveleteket a `Directory` osztályon keresztül érjük el. Könyvtárat hozhatunk létre a **CreateDirectory** és törölhetünk a **Delete** metódusokkal. Utóbbinál második paraméterként `true` értéket adva annak teljes tartalmát törölni tudjuk, minden benne lévő könyvtárral és fájllal együtt. Egy könyvtár létezését ellenőrizhetjük az **Exists** metódussal. Egy adott elérési útvonalon lévő könyvtár összes alkönyvtárát a **GetDirectories** metódussal, a benne lévő összes fájlt a **GetFiles** metódussal kérhetjük le, utóbbinál második paraméterként egyszerű szűrőfeltétel is megadható a `*` vagy a `?` karakter használatával (pl. `"*.txt"`, ami minden `.txt` kiterjesztésű fájlt visszaad). A metódusok első paramétereként megadott útvonal minden esetben egy `string`, amelyben kikapcsolhatjuk a speciális karakterek értelmezését a karakterlánc előtt elhelyezett `@` jellel (ellenkező esetben az elérési útvonalban gyakran szereplő \ karakter hibát okozna, így azokat minden előfordulás esetén escapelni kellene).

```
using System.IO;
namespace FilesAndDirectories
{
    class Program
    {
        static void Main(string[] args)
        {
            Directory.CreateDirectory(@"C:\MyFolder\MyFirstFolder");
            Directory.CreateDirectory("C:\\MyFolder\\MySecondFolder");
            string[] allFolders = Directory.GetDirectories(@"C:\MyFolder");
            // { "C:\\MyFolder\\MyFirstFolder", "C:\\MyFolder\\MySecondFolder" }
            string folderToDelete = "C:\\MyFolder";
            if (Directory.Exists(folderToDelete))
            {
                Directory.Delete(folderToDelete, true);
            }
        }
    }
}
```

Fájlok kezelése

Fájlok kezeléséről kétféle értelemben beszélhetünk: *fájlrendszer szintű kezelés* alatt értjük a fájlok létrehozását, másolását, törlését, adataik lekérdezését, míg *fájlkezelés* alatt egy konkrét fájl tartalmának feldolgozását vagy módosítását.

A `File` osztály metódusai mind fájlrendszer szintű kezelést, mind fájlkezelést lehetővé tesznek. A **Copy** metódus az első paraméterként megadott forrásfájlt másolja a második paraméterként megadott célfájlra. A **Move** működése hasonló, de ez áthelyezi a forrásfájlt. A **Delete** a paraméterben megadott elérési úton lévő fájlt törli. Egy adott útvonalon feltételezett fájl létezését az **Exists** metódussal tudjuk ellenőrizni.

```
string original = @"C:\SourceFolder\mySource.txt";
string copy = @"C:\TargetFolder\myTarget.txt";
File.Copy(original, copy);
if (File.Exists(original))
{
    File.Delete(original);
}
File.Move(copy, original);
```

Egy szöveges fájl sorait, valamint teljes tartalmát beolvashatjuk a **ReadAllLines** és a **ReadAllText** metódusokkal. Szöveges fájlhoz fűzhetünk további sorokat vagy tartalmat az **AppendAllLines** és az **AppendAllText** metódusokkal. Amennyiben új fájlba szeretnénk sorokat vagy tartalmat írni (vagy felül kívánjuk írni a már benne lévő tartalmat), használhatjuk a **WriteAllLines** és a **WriteAllText** metódusokat. Minden esetben megadható paraméterként a felhasznált kódolás is, ami .NET keretrendszer használata esetén az UTF-8 (egyéb esetben a `System.Text.Encoding` valamely értéke használható).

```
string fileName = "heisenberg.txt";
string text = "You clearly don't know who you're talking to, so let me clue you
    in.\nI am not in danger, Skyler.\nI am the danger.";
string[] lines = { "A guy opens his door and gets shot, and you think that of
    me?", "No!", "I am the one who knocks!" };
File.WriteAllText(fileName, text, System.Text.Encoding.ASCII);
File.AppendAllLines(fileName, lines);
string quoteAsText = File.ReadAllText(fileName);
// "You clearly don't know who you're talking to, so let me clue you in.\n...\nI
    am the one who knocks!\n"
string[] quoteAsArray = File.ReadAllLines(fileName);
// { "You clearly don't know who you're talking to, so let me clue you in.", ...
    "I am the one who knocks!" }
```

Fájlok mint folyamok

Lehetőségünk van fájlokat folyamként (stream) kezelni. Ez azt jelenti, hogy egy szöveges fájl tartalmát képzelhetjük karakterek „időben változó sorozatának", amelynek feldolgozása hatékonyabb lehet például memóriafoglalás szempontjából, hiszen nem feltétlenül szükséges egy fájl teljes tartalmát egyszerre a memóriában tartanunk, hanem feldolgozhatjuk azt részletenként is. Megjegyezzük, hogy a `File` osztály egyébként szintén az alábbi két osztályt használja a fájlok feldolgozása során.

Szöveges fájlok folyamként írását teszi lehetővé a `StreamWriter` osztály. Az íráshoz használt objektum létrehozásakor adjuk meg a célfájl elérési útvonalát. Opcionálisan megadható még, hogy már létező fájl esetén hozzáfűzés történjen-e (`true`), továbbá beállítható a kódolás is a fentebb bemutatott módon. A **Write** és **WriteLine** metódusok a konzolnál megismertekhez hasonlóan működnek (vagyis a **WriteLine** sortörés karakterrel zárja a paraméterként kapott szöveget). A folyamat végén ne felejtsük el bezárni a folyamot a **Close** metódussal, amely leüríti a folyam által használt puffert (ennek hiányában elképzelhető, hogy a kiírt adatok egy része nem jelenik meg a fájlban, illetve nyitott fájlhoz más folyamat csak korlátozottan férhet hozzá)!

```
StreamWriter myWriter = new StreamWriter("elcamino.txt", true);
myWriter.WriteLine("- What kind of pizza do you like, Jesse?");
myWriter.WriteLine("- Pepperoni.");
myWriter.Write("- Pepperoni, sure, classic. ");
myWriter.WriteLine("I like that too.");
myWriter.Close();
```

Egy szöveges fájlt folyamkét olvashatunk a `StreamReader` osztállyal. Az objektum létrehozásakor adhatjuk meg a fájl elérési útvonalát, valamint opcionálisan a kódolását. Az `EndOfStream` tulajdonság megadja, hogy van-e még olvasható adat a folyamban (másképp fogalmazva, hogy a fájl végére értünk-e az olvasással). Az alábbi kódrészlet a megnyitott fájlból addig olvas egy-egy sort a **ReadLine** metódussal, amíg a folyamot képező fájl végére nem ér.

```
StreamReader myReader = new StreamReader("elcamino.txt",
    System.Text.Encoding.ASCII);
while (!myReader.EndOfStream)
{
    string oneLine = myReader.ReadLine();
    Console.WriteLine(oneLine);
}
myReader.Close();
```

Bináris fájlok feldolgozás folyamként

Nem minden adatot célszerű szöveges formában tárolnunk, mivel ilyenkor a tárigény sokszorosa lehet a valóban szükségesnek. Például az 1234567890 egész szám szöveges formában tárolásához 10 karakter (összesen 20 bájt) szükséges, miközben az érték gond nélkül ábrázolható egy 32 bites (4 bájtos) egészként is. A `BinaryReader` és `BinaryWriter` osztályok segítségével bármilyen C# alaptípust (primitívet) bináris formában olvashatunk vagy írhatunk. A két osztály használata némileg hasonló a már bemutatott `StreamReader` és `StreamWriter` osztályokéhoz, ezért a konkrét metódusok részletes ismertetése helyett csak egy példakódot mutatunk néhány alaptípusú érték fájlba írásáról és visszaolvasásáról. Eltérés az előbbiekhez képest, hogy nem tudjuk közvetlenül a `BinaryReader` és `BinaryWriter` példányosításakor megadni a fájl elérési útvonalát, hanem előbb egy `FileStream` objektumot kell létrehoznunk, amely a bináris fájlt reprezentálja. Fontos, hogy beolvasáskor helyes sorrendben értelmezzük a fájlban lévő adatokat, ellenkező esetben az eredetitől eltérő értéket kaphatunk eredményül bármilyen hibaüzenet nélkül! A példa egy ilyen esetet mutat be.

```
byte oneByte = 1;
bool oneBoolean = false;
char oneChar = 'A';
double oneDouble = 1.0;

FileStream writerStream = new FileStream("somedata.bin", FileMode.Create);
BinaryWriter myWriter = new BinaryWriter(writerStream);
myWriter.Write(oneByte);
myWriter.Write(oneBoolean);
myWriter.Write(oneChar);
myWriter.Write(oneDouble);
myWriter.Close();
writerStream.Close();

FileStream readerStream = new FileStream("somedata.bin", FileMode.Open);
BinaryReader myReader = new BinaryReader(readerStream);
bool oneBooleanRead = myReader.ReadBoolean();       // true
double oneDoubleRead = myReader.ReadDouble();       // 8.2212523467983425E-320
byte oneByteRead = myReader.ReadByte();             // 240
char oneCharRead = myReader.ReadChar();             // '?'
myReader.Close();
readerStream.Close();
```

OOP alapjai: osztályok szerkezete

Összefoglaló

Az objektumorientált programozás (OOP) egy olyan módszertan (paradigma), amely a logikailag összetartozó adatokat és a rajtuk végezhető műveleteket úgynevezett *objektumokba* csomagolja, ez az *egységbezárás* alapelve. Gondoljunk például egy egyszerű bankalkalmazásra: az ügyfeleket, a bankszámlákat és a tranzakciókat modellezhetjük objektumokként. Az ügyfelek egy osztályt alkothatnak, amely tartalmazza az ügyfél nevét, címét és számlaszámát (ezek a logikailag az ügyfélhez tartozó adatok). A bankszámlák szintén osztályként definiálhatók, és tartalmazhatják az aktuális egyenleget és tranzakciókat, a tranzakciók pedig további objektumok lehetnek, amelyek tartalmazzák az összeget és a dátumot. Minden objektumnak van egy meghatározott szerepe és felelőssége: az ügyfél például pénzt helyezhet be vagy veszhet le a bankszámlájáról (ez az objektum adatain végzett műveleteket jelenti), a bankszámla számítja az egyenleget és rögzíti a tranzakciókat, míg a tranzakció egyszerűen rögzíti az összeget és a dátumot.

Nézzük meg az előbbi példában szereplő *ügyfél* egy lehetséges modelljét, amely csak az adatait tárolja (tehát műveleteket egyelőre nem valósít meg).

```
namespace BankApp
{
    class BankClient
    {
        public string name;
        string userID;
        private DateTime dateOfBirth;
        public double balance;
    }

    class Program
    {
        static void Main(string[] args)
        {
        }
    }
}
```

Látható, hogy a `BankClient` osztály elkülönül a **Main** metódust tartalmazó osztálytól. A `BankClient` ebben a példában négy *tagváltozót* vagy *adatmezőt* tartalmaz, amelyek lényegében közönséges változók, az egyetlen speciális jellemzőjük, hogy mind ugyannak az osztálynak a változói, vagyis egyetlen egység részei. Az osztály tehát a banki ügyfelek egyféle sablonja: nem azt definiálja, hogy pontosan mi kell legyen az ügyfél neve, azonosítója, születési dátuma vagy egyenlege, hiszen ez minden ügyfélnél különböző lehet, de előírja azt, hogy minden ügyfélnek rendelkeznie kell ezekkel a jellemzőkkel.

Példányosítás

Az alábbi kódrészletben két *példányt* (vagy más néven objektumot) készítünk el, a példányosításhoz a `new` operátort kell használnunk (ahogy tettük azt például a `Random` osztály esetében is). A `clientOne` és `clientTwo` nevű változókon keresztül egy-egy olyan példányt érünk el ezt követően, amelyeknek saját neve, azonosítója, születési dátuma és egyenlege van. Ami közös bennük, hogy mindketten ugyanabból az osztályból (sablonból) lettek példányosítva.

```
static void Main(string[] args)
{
    BankClient clientOne = new BankClient();
    BankClient clientTwo = new BankClient();
}
```

Taghozzáférés

Lehetőségünk van a példányok tagjainak (mezőinek és metódusainak) külső elérésére, amennyiben azokat az osztályban elláttuk a `public` módosítóval. Amely tag előtt nem szerepel ez, azok belső (privát) tagok, és kívülről nem hozzáférhetők. (Ezt az oszályban külön is jelezhetjük a `private` kulcsszó kiírásával.)

```
BankClient clientOne = new BankClient();
BankClient clientTwo = new BankClient();

clientOne.name = "Mike Ehrmantraut";
clientTwo.name = "Gustavo Fring";
clientTwo.balance = 123456789.0;
clientOne.userID = "MIKE123"; // !!!  INACCESSIBLE !!!
Console.WriteLine(clientTwo.balance);
```

Tagfüggvények (metódusok)

Az osztályban az adatmezőkön felül úgynevezett *tagfüggvényeket* vagy metódusokat is definiálhatunk, amelyek utasításokat tartalmaznak. A metódus *meghívása* ezen utasítások lefuttatását jelenti, és ennek segítségével tudunk például az objektum tagváltozóival dolgozni (beállítani, lekérdezni, módosítani), amely megfeleltethető a bevezetőben említett műveletvégzésnek. Az alábbi osztályban két metódust definiálunk.

A **SetName** metódus egy karakterláncot kér paraméterként, és annak megfelelően állítja be a példány adattagjának értékét. Ettől a metódustól nem várunk *visszatérési értéket*, ezt a neve elé írt `void` kulcsszóval kell jeleznünk. Mivel azt szeretnénk, hogy a metódust külső programrészek is elérjék, így a láthatóságát publikusra állítjuk.

A **GetFirstnameAndLastname** metódus paraméter nélküli (vagyis meghíváskor nem kell argumentumként valamilyen adatot átadnunk neki). A metódus egy karakterlánc tömböt ad vissza (ezt jelezünk kell a neve előtt a típus megadásával), majd a metódus *törzsében* (tipikusan utolsó utasításként) a `return` kulcsszóval egy előírtnak megfelelő típusú értéket kell visszaadnunk.

Az osztályban ilyen módon definiált metódusokat az osztályból készített példányon keresztül tudjuk meghívni.

```
class BankClient
{
    private string name;

    public void SetName(string clientName)
    {
        name = clientName;
    }

    public string[] GetFirstnameAndLastname()
    {
        string[] firstAndLast = name.Split(' ');
        return firstAndLast;
    }
}

class Program
{
    static void Main(string[] args)
    {
        BankClient clientOne = new BankClient();
        BankClient clientTwo = new BankClient();

        clientOne.SetName("Eduardo Salamanca");
        clientTwo.SetName("Nacho Varga");
        string[] clientOneFullName = clientOne.GetFirstnameAndLastname();
        // { "Eduardo", "Salamanca" }
    }
}
```

Konstruktor

Egy osztály konstruktora egy speciális metódus az osztályban a példányok elkészítésére. A konstruktor feladata az objektum mezőinek inicializálása, vagyis azok kezdeti értékekkel történő feltöltése. A konstruktor felépítése hasonló az előbb említett metódusokéhoz, vagyis rendelkezhet paraméterekkel, azonban neve kötelezően az osztály nevével kell megegyezzen, és visszatérési értéket nem írhatunk elő. Az alábbi kódrészletben az oszály kétparaméteres konstruktora az átadott értékeknek megfelelően állítja be az ügyfél nevét és azonosítóját, a számlaegyenleg azonban minden esetben 1.0 kezdőértéket kap.

Gyakori, hogy egy metódus valamely paraméterének ugyanaz a neve, mint az osztály egyik tagváltozójának. Ilyen esetben különbséget tehetünk a két szimbólum között azzal, hogy a mező neve elé írjuk a `this` kulcsszót. A kulcsszó a program futtatásakor azt a példányt reprezentálja, amelyen keresztül a taghozzáférést kezdeményeztük. Az alábbi példában ez azt jelenti, hogy az első konstruktorhíváskor az átadott azonosító a `clientOne` változóban tárolt példány `userID` mezőjének értéke lesz, a második konstruktorhíváskor az átadott érték viszont értelemszerűen a `clientTwo` példány mezőjében kerül eltárolásra. A `this` kulcsszó egyértelműsítés céljából egyébként máskor is kiírható a tagok elé.

```
class BankClient
{
    private string name;
    private string userID;
    private double balance;

    public BankClient(string clientName, string userID)
    {
        name = clientName;
        this.userID = userID;
        this.balance = 1.0;
    }
}

class Program
{
    static void Main(string[] args)
    {
        BankClient clientOne = new BankClient("Eduardo Salamanca", "LAL042");
        BankClient clientTwo = new BankClient("Hector Salamanca", "TIO1939");
    }
}
```

Láthatósági szabályok

Az egységbezárás alapelvéhez szorosan kapcsolódik az *adatrejtés* alapelve, amely azt a célt szolgálja, hogy az objektumok állapotát a „külvilágból" csak erre definiált metódusokon keresztül lehessen módosítani, vagyis csak közvetett módon lehessen elérni őket. Az elv alkalmazásával megakadályozhatjuk az érvénytelen állapotok létrehozását az objektumokban és kivédhetjük az olyan hibákat, amelyek akkor történhetnek, amikor az objektumok adatait közvetlenül módosítják az osztályon kívülről.

Ennek megfelelően az osztály semelyik mezőjének láthatóságát sem javasolt a `public` kulcsszóval nyilvánossá tenni, továbbá az osztály tagfüggvényei közül is csak azokat javasolt publikusnak jelölni, amelyek külső elérése mindenképp szükséges.

Az alábbi példában a `BankClient` osztály minden mezője privát láthatóságú (mivel ez az alapértelmezett), vagyis kívülről nem érhetőek el. Ettől függetlenül, mivel az osztály részei, így az osztály minden tagfüggvénye eléri őket. A belső használatra létrehozott **HasPositiveBalance** segédmetódust az osztályon kívülről nem tudjuk meghívni, azonban az osztály bármely metódusa meghívhatja, például a **GetClientStatus** metódus is. Figyeljük meg, hogy az osztály metódusai hozzáférnek az osztály tagjaihoz (tagváltozókhoz és tagfüggvényekhez is), ezen felül pedig szükség esetén paramétereket is előírhatunk, így meghívásukkor az oszályon kívüli adatokkal is el tudjuk látni őket.

```
class BankClient
{
    string userID;
    double balance;

    public BankClient(string userID)
    {
        this.userID = userID;
        this.balance = 1.0;
    }

    private bool HasPositiveBalance()
    {
        bool positiveBalance = this.balance > 0;
        return positiveBalance;
    }

    public string GetClientStatus(DateTime currentDate)
    {
        if (this.HasPositiveBalance())
        {
            return $"UserID {userID} has positive balance at {currentDate}";
        }
        else
        {
            return $"UserID {userID} does not have a positive balance at
               {currentDate}";
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        BankClient tio = new BankClient("TIO1939");
        string status = tio.GetClientStatus(DateTime.Now);
        // UserID TIO1939 has positive balance at 2023. 10. 23. 20:23:10
    }
}
```

Összefoglaló

Osztályok közötti kapcsolatok

A programunkban általában több osztályt is deklarálunk, amelyek között különféle kapcsolatokat hozunk létre. Egy ilyen kapcsolat az osztályok *függősége*, amikor egy osztály valamely példánya használja egy másik osztály példányát. Azt mondjuk, hogy egy osztály függ egy másiktól, ha a független osztályunk paraméterként vagy lokális változóként megjelenik a függő osztály valamely metódusánál. Erre mutat példát az alábbi programkód, ahol a `Player` osztály valamely példányán keresztül meghívott **UseItem** metódusnak paraméterként egy `Item` típusú objektumot kell átadnunk.

```
class Item
{
    public void DoSomething() {   }
}
class Player
{
    string name;
    int score;

    public Player(string name) {   }

    public void UseItem(Item item) {
        item.DoSomething();
    }
}
```

Asszociáció típusú kapcsolatról beszélünk, amikor a független osztály példánya a függő osztály attribútumaként (mezőjeként) jelenik meg. Az alábbi példában a `Game` osztály az előbbi `Player` osztályból példányosított objektumokat tárol egy listában (a játék résztvevőit), tehát a `Player` típus a `Game` osztály attribútumaként van jelen.

```
class Game
{
    List<Player> players;

    public Game() {
        players = new List<Player>();
    }
}
```

Megemlítjük még az asszociációs kapcsolat speciális esetét, az *aggregációt*, amikor az egyik osztály objektumai részét képezik egy másik osztály objektumainak. *Kompozíció* jellegű aggregációról beszélünk, amikor a függő és független objektum életciklusa megegyezik (vagyis egyszerre jönnek létre, és egyszerre is szűnnek meg). További

speciális kapcsolat az osztályok között a *származtatás*, amelyet ezen tárgy keretein belül nem tárgyalunk.

Metódusok túlterhelése

Egy metódust a neve és a paramétereinek listája egyértelműen azonosít, ezeket együtt a metódus *szignatúrájának* nevezzük[5]. Lehetőségünk van egy már létező metódushoz eltérő implementációt (új működést) adnunk, amennyiben a két metódus paraméterlistáját különbözőnek definiáljuk. Ilyen esetben a metódus *túlterheléséről* (overloading) beszélünk, és meghíváskor az átadott paraméterek típusa és sorrendje alapján dől el, melyik megvalósítás fog lefutni. Az alábbi példában a `Point` osztály **Show** metódusa három különböző módon hívható meg: paraméter nélkül, egyetlen karakterrel, illetve egy karakterrel és egy színnel paraméterezve.

Természetesen lehetőségünk van valamely metódusban egy másik metódus, illetve egy túlterhelt változat hívására is, amely hasznos lehet olyan esetekben, amikor el szeretnénk kerülni bizonyos kódsorok ismételt leírását, ha azt egy másik metódus már megvalósította. Erre mutat egyszerű példát a **Show** metódus kétparaméteres változata.

```
class Point
{
    int positionX;
    int positionY;

    public void Show()
    {
        Console.SetCursorPosition(positionX, positionY);
        Console.Write('*');
    }
    public void Show(char symbol)
    {
        Console.SetCursorPosition(positionX, positionY);
        Console.Write(symbol);
    }
    public void Show(char symbol, ConsoleColor color)
    {
        Console.ForegroundColor = color;
        Show(symbol);
    }
}
```

A metódusok túlterhelésének speciális esete a konstruktorok túlterhelése. Amennyiben egy konstruktor már megvalósít bizonyos működést, amelyet szeretnénk kihasználni egy másik konstruktorban, akkor a metódusoknál megismertektől eltérően *konstruktor hívási láncot* kell létrehoznunk az alábbi példához hasonló módon a `this` kulcsszó használatával. Ilyen esetekben az elsőként hívott konstruktor törzsében felsorolt utasítások végrehajtása előtt a másodikként megjelölt konstruktor fut le előbb, ezt követi csak az elsőként hívottban felsorolt utasítások végrehajtása.

Az alábbi példában egy személyt létrehozhatunk kizárólag a neve megadásával (ilyenkor minden más mező értékét a típusnak megfelelő alapértékre állítja a konstruktor). Létrehozhatjuk ugyanakkor a személyt a neve, magassága, testtömege és testtömeg-indexe megadásával is, a név beállítását azonban elvégeztethetjük a korábban már megírt

[5] Kontextustól függően a szignatúrának része a visszatérési érték típusa is, lásd bővebben itt: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/methods#method-signatures

egyparaméteres konstruktorral is. Hasonlóan, a testtömeg-index kiszámítása a magasság és testtömeg alapján könnyen megoldható (vagyis ezt nem feltétlenül kell külön paraméterként elkérnünk), tehát a három paraméterrel rendelkező konstruktor meghívhatja a négyparaméteres változatot a mezők inicializálásához.

```
class Person
{
    string name;
    double height;
    double weight;
    double BMI;

    public Person(string name)
    {
        this.name = name;
    }

    public Person(string name, double height, double weight, double BMI) :
       this(name)
    {
        this.height = height;
        this.weight = weight;
        this.BMI = BMI;
    }

    public Person(string name, double height, double weight) : this(name,
       height, weight, weight/(height*height))
    {
    }
}
```

Tulajdonságok

Az egységbezárás elvének megfelelően egy objektum mezői privát elérésűek, vagyis külső osztályok számára közvetlenül nem hozzáférhetőek. Az objektum adatainak lekérdezése és módosítása épp ezért általában metódusokon keresztül történik. A C# nyelv ennek a folyamatnak a megkönnyítésére úgynevezett *tulajdonságok* (properties) használatát kínálja fel. A tulajdonságok speciális tagok az osztályokban, amelyek lehetővé teszik a mezők kontrollált elérését és beállítását: olyan metódusokról van szó, amelyek használat során mezőként viselkednek.

Egy tulajdonság általában két részből áll: a **get** rész az olvasásért (adatlekérésért), míg a **set** rész az írásért (adatbeállításért) felel. Az alábbi példában a `Circle` osztály egyetlen privát adatmezőt deklarál, amelyhez a tulajdonságon keresztül férhetünk hozzá más osztályokból. Szokás a tulajdonság nevét a kapcsolódó mező nevével egyezőnek választani, és nagy kezdőbetűvel ellátni. A **get** részben lévő kódrészlet ebben az esetben közvetlenül a háttérben lévő mező értékét szolgáltatja vissza. A **set** részben a beállítani kívánt értékre (ez a **Main** metódusban 2.5 lesz) a `value` kulcsszóval hivatkozhatunk. A példában a kör sugarának beállítása csak pozitív érték esetén történik meg, így megakadályozhatjuk a téves értékadásból származó hibás működést.

Lehetséges csak **get** vagy csak **set** részek deklarálása is, illetve ezekben a részekben akár komplexebb kódrészletet is megadhatunk. Erre mutat példát az osztály **Area** tulajdonsága, amelyen keresztül a kör területét kérdezhetjük le (számítás eredményeként, nem közvetlenül egy mező értékét visszaadva), azonban mivel **set** részt nem hoztunk

létre, így a terület beállítása a tulajdonságon keresztül nem lehetséges. A **get** és **set** részek hozzáférhetősége szükséges esetben szigorítható az eléjük írt módosító kulcsszó használatával, így elérhető például, hogy egy deklarált **set** rész csak az osztályon belül legyen hozzáférhető, míg a **get** rész publikus más osztályok számára is.

```
class Circle
{
    double radius;
    public double Radius
    {
        get { return radius; }
        set {
            if (0 < value)
            {
                radius = value;
            }
        }
    }
    public double Area
    {
        get { return Math.Pow(radius, 2) * Math.PI; }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Circle circle = new Circle();
        circle.Radius = 2.5;
        Console.WriteLine($"Area of the circle is {circle.Area}.");
        circle.Area = 10; // !!!  READ ONLY !!!
    }
}
```

Az úgynevezett *automatikusan megvalósított tulajdonságok* (Auto-Implemented Properties, AIP) egyszerű és rövid tulajdonságdeklarációt tesznek lehetővé, és olyan esetekben előnyösek, amikor a mezők hozzáférésekor nincs szükség komplexebb logika megvalósítására. AIP használatakor a fordító egy privát, névtelen mezőt hoz létre a háttérben, amelyet csak a tulajdonság **get** és **set** tagjain keresztül lehet elérni.

```
class ComplexNumber
{
    public double RealPart { get; set; }
    public double ImaginaryPart { get; set; }
}
```

Felsorolt típus (enum)

Felsorolt típus alatt előre definiált értékek (címkék) halmazát értjük. Ilyen típus használata célszerű lehet, amikor olyan adatokat akarunk tárolni, amelyeknek csak egy meghatározott számú értékük lehet (például napok nevei,

vagy az iskolai értékelésnél használt osztályzatok megnevezései). Saját felsorolt típust az enum kulcsszóval, a típus nevének, valamint lehetséges értékeinek megadásával definiálhatunk az alábbihoz hasonló módon.

```
enum Days { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }
// Monday=0, Tuesday=1, Wednesday=2, ...
```

A címkéket egész értékek azonosítják, amelyek egyénileg beállíthatók. Ha nem rendelkezünk másképp, a felsorolt típus első címkéje a 0, a következő az 1, stb. értékeket kapja.

```
enum Days { Monday=3, Tuesday, Wednesday, Thursday, Friday=420, Saturday, Sunday
    }
// Monday=3, Tuesday=4, ... Friday=420, Saturday=421, ...
```

Egy felsorolt típusú változó értékei ezt követően csak az előre definiált értékek közül kerülhetnek ki.

```
Days myDay = Days.Friday;
```

A felsorolt értékek egész értékké konvertálhatók, illetve megfelelő egész értékeket felsorolt értékekké konvertálhatunk az alábbihoz hasonló módon.

```
Days myDay = Days.Friday;
int fridayAsInt = (int)myDay;   // 420
int sundayAsInt = 422;
Days anotherDay = (Days)sundayAsInt;    // Days.Sunday
```

Lehetőségünk van a felsorolt típus értékét karakterlánccá alakítani a megszokott módon, a **ToString** metódus használatával.

```
Days myDay = Days.Friday;
string myDayString = myDay.ToString();   // "Friday"
```

Leteséges karakterláncokból is kinyerni a címkék értékét az Enum osztály **Parse** nevű metódusával. A metódus első paramétere az a *típus*, amelynek az értékeit kívánjuk beolvasni, második paramétere a feldolgozandó karakterlánc. A metódus által visszaadott értéket utolsó lépésként át kell alakítanunk a kívánt felsorolt típusba, az alább látható módon.

```
Days myDay = (Days)Enum.Parse(typeof(Days), "Friday");   // Days.Friday
```